Homework 2

William Flanders*
Haverford College
(Dated: February 9, 2023)

1. NEWMAN EXERCISE 2.13A

For Exercise 2.13a, we want to create a python program that can return the nth Catalan number. The way we can achieve this is through recursion. Recursion is a form of calculation where we utilize previous values to find target values. An example is the factorial,

$$n! = n * (n-1) * (n-2) * (n-3) * ... * 1. \tag{1}$$

As we see in Equation 1, the target value of a factorial is reliant on previous values. We can create a python program to model this,

```
def factorial(n):
    if n == 1:
        return 1
    else:
        return factorial(n-1)*n
```

As we can see in 1, our target factorial value is reliant on a recursive call of the factorial function if n ¿ 1.

We want to use this technique to a create a program for the Catalan numbers. They can be defined mathematically as

$$C_n = \begin{cases} 1 & \text{if } n = 0, \\ \frac{4n-2}{n+1} C_n & \text{if } n > 0. \end{cases}$$

We can see that, just like for 1, we will need two cases. One case for when $n = 0$ and the recursive case for when $n > 0$. We begin with pseudo code to map our logic,

```
def catalan(n):
    if n = 0:
        return 1
    else if n > 0:
        return (4 times n-2 divded by n+1)
        times the n-1th Catalan number
```

Once we have our program, we can compute the 100th Catalan number. This value is $8.965 \cdot 10^{56}$.

*Electronic address: wflanders@haverford.edu

2. NEWMAN EXERCISE 3.1

Provided a txtfile that lists months in one column and number of sunspots in the other, we want to be able to plot sunspots against time. To do this, we need to take advantage of the python library matplotlib. Matplotlib comes equipped with a number of useful plotting tools, one of which is the scatter function. The scatter function takes in data for x and y and forms a scatter plot between the two. We start with pseudo code,

```
import matplotlib.pyplot as plt
import numpy as np

data = load sunspots.txt
x = data[1st column]
y = data[2nd column]

plt.scatter(x, y)
plt.show()
```

We can use the library numpy to load in the data from our sunspots.txt file. This is done through the loadtxt function which takes a txt file as the argument. Once we have our program, we can use it to generate a plot as seen in Figure 1.

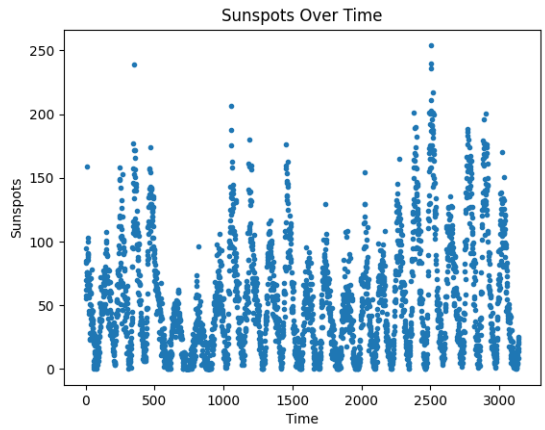

FIG. 1: Sunspots over time from 0 to 250 Years

What is most notable about the plot is the cyclical behavior of the number of sunspots. We can see that the number of sunspots is never constant and instead rises and falls over time. Some months there are nearly 0 sunspots while in other months there are 250+.

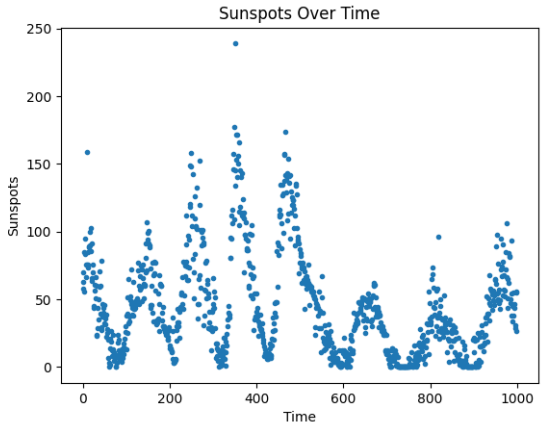

FIG. 2: Sunspots over time from 0 to 83.33 years

We can modify our program to only display the first 1000 months. This provides us a zoomed in view. The edits to the code are fairly simple. With our x variable, we can simply slice it such that we only have the first 1000 entries and store them in a new list. The same can be done for the y variable,

```
new_x = x[:1000]
new_y = y[:1000]
```

Once we've done this, running the program again generates a plot seen in 2,

From the figure, we can see that once zoomed in over a time span of a thousand months, the cyclical nature of the number of sunspots is much more visible. It appears that the number of sunspots follows a period of roughly 10-15 years where it rises dramatically, peaks at a hundred to two hundred, before falling back to nearly zero again.

We can look to calculate a running average over the data in 1000 month time span to get an idea of how the number of sunspots are changing. The formula for a running average is given by

$$Y_k = \frac{1}{2r+1} \sum_{m=-r}^{r} y_{k+m} \tag{2}$$

where r = 5. This parameter determines the look back period for our running average. In essence, we will be averaging the 5 data points prior to the target and 5 data points after. We will want to plot this running average. To do this, we can again utilize matplotlib as well as the average function from numpy. Beginning with pseudo code:

```
import numpy
import matplotlib

x = first 1000 months
y = first 1000 elements of
    number of sunspots

running_av = list full of zeroes
             the length of y

for i in range(len(running_av)):
    rolling_av[i] = np.average(
                    y[the 5 prior data points
                    and 5 later data points])

plt.scatter(x, y)
plt.plot(x, rolling_av)
plt.show()
```

With our program, we can now over layer our rolling average on top of our scatter plot for the period from 0 months to 1000 months. The plot is show in Figure 3.

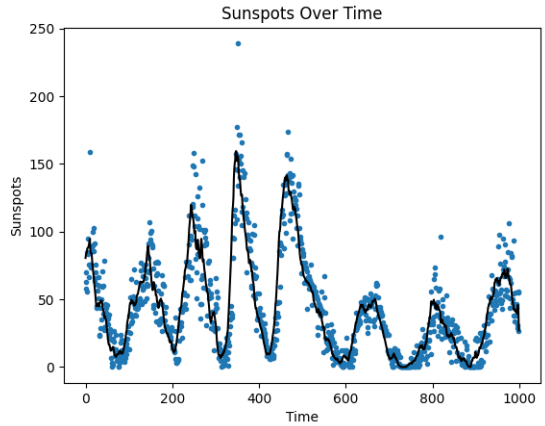

FIG. 3: A rolling average of sunspots over time layered on top of a scatter plot

It is much easier to make out the structure and periodic up and downs in the number of sunspots when the running average is layered on top of our scatter plot. We can see two large peaks, indicating that the number of sunspots in those two periods was abnormally high. Interestingly, we can see two very low peaks just after the two large peaks. This seems to indicate that while the average number of sunspots rose in the previous two periods, it was counterbalanced by a decrease in the average number of sunspots in the next two periods.

3. NEWMAN EXERCISE 3.2

Plotting polar functions in python is more than possible. The trick relies in converting from polar coordinates

(r, θ), to Cartesian coordinates (x, y). This can be done through the formulae

$$x = r\cos\theta \tag{3}$$

for the x coordinate and

$$y = r\sin\theta \tag{4}$$

for the y coordinate. For example, the deltoid curve can be defined parametrically as

$$x = 2\cos\theta + \cos 2\theta \tag{5}$$

$$y = 2\sin\theta - \sin 2\theta \tag{6}$$

where $0 \leq \theta \leq 2\pi$. We can write a python program to plot this curve using numpy and matplotlib. Starting with pseudo code,

```
import numpy as np
import matplotlib.pyplot as plt

theta = np.linspace(0, 2pi)

x = Equation 5
y = Equation 6

plt.plot(x,y)
plt.show()
```

An important tool is the numpy linspace function. It allows for an interval to be created, in this instance from 0 to 2π, with a set step size. For this curve, we only need the default which is set to 50. Figure 4 shows the plotted function.

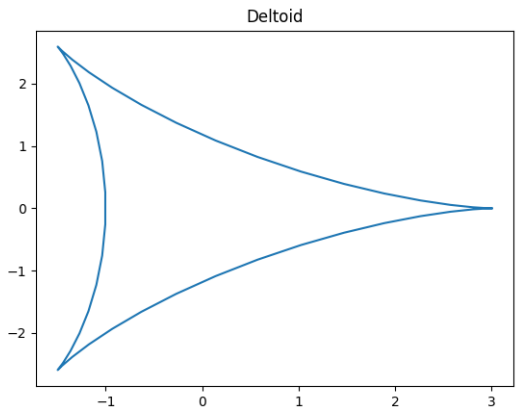

FIG. 4: Deltoid Curve

Another function that we can plot is the Galilean spiral. It can be formed from Equations 3 and 4 where $r = \theta^2$ and $0 \leq \theta \leq 10\pi$.

For our program, we can plan with similarly designed pseudo code

```
import numpy as np
import matplotlib.pyplot as plt

theta = np.linspace(0, 10pi, 1000)
r = theta**2

x = Equation 3
y = Equation 4

plt.plot(x,y)
plt.show()
```

and output Figure 5 for our plot. In this instance, we've increased a step size to increase the plot's smoothness.

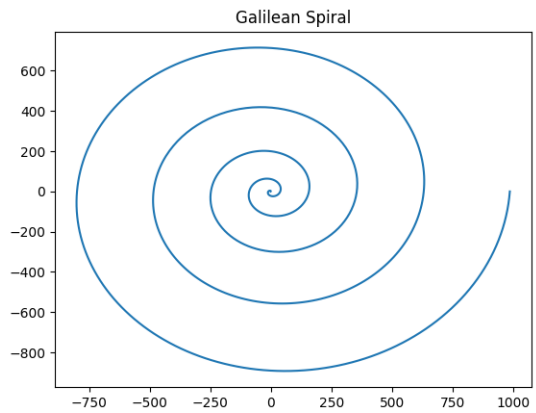

FIG. 5: Galilean Spiral

Finally, it doesn't matter the complexity of the function that we are plotting. Take Fey's function, which has a complicated value for r,

$$r = e^{\cos\theta} - 2\cos 4\theta + \sin^5\frac{\theta}{12} \tag{7}$$

where $0 \leq \theta \leq 24\pi$. This is easily substituted into Equations 3 and 4 for our python program. For our pseudo code,

```
import numpy as np
import matplotlib.pyplot as plt

theta = np.linspace(0, 24pi, 1000)
r = Equation 7

x = r * Equation 3
y = r * Equation 4

plt.plot(x,y)
plt.show()
```

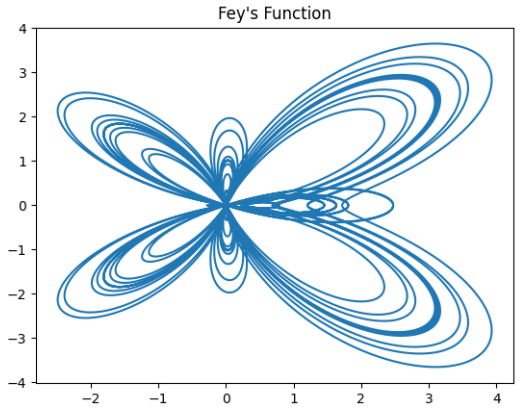

FIG. 6: Fey's Function

Figure 6 shows the plot of this function.

Similarly, just as we needed to increase the step size for the Galilean spiral, we also need to increase the step size for Fey's function in order to retain its smoothness.

4. SURVEY

The assignment took roughly 7 hours. Like last time, the most difficult part was the LaTeX typesetting and getting everything to fit nicely within the .tex file. The running average portion for Exercise 3.1 was somewhat challenging but otherwise the coding was enjoyable.